1 — DÉFINITIONS

Un **algorithme** est une **suite d'instructions élémentaires** qui s'appliquent **dans un ordre déterminé** pour fournir un résultat (exemple : suivre une recette de cuisine).

Un **algorigramme** est la **représentation graphique** d'un algorithme à l'aide de **symboles normalisés**. Il permet de représenter et d'étudier le fonctionnement d'un système.

Exemple : afficher le carré des nombres de 1 à 10

```
DÉBUT Calcul
  Pour A de 1 à 10 par pas de 1
    B ← A²
    Afficher B
  Fin Pour
FIN Calcul
```

Lire l'algorithme

« B ← A² » se lit « B reçoit A² » : on **calcule** A² puis on le **range** dans la variable B (affectation).

La boucle **Pour** répète les deux instructions 10 fois, avec A = 1, 2, … 10. L'algorigramme de cet exemple est donné au § 4.

2 — SYMBOLES NORMALISÉS

Symbole	Désignation et rôle	Symbole	Désignation et rôle
Début / Fin	**Début / fin** début ou fin de l'algorithme	Action	**Traitement** opération sur des données : calcul, affectation, action
Lire / Écrire	**Entrée / sortie** **lecture** d'une donnée (capteur, clavier) ou **écriture** (affichage, commande d'une sortie)	Test ? NON	**Test (branchement conditionnel)** décision : on choisit entre **deux chemins** selon une condition ; sorties **OUI** et **NON**
Sous-prog.	**Sous-programme** appel d'un sous-programme décrit ailleurs	↓	**Liaison** relie les symboles ; le parcours se fait **de haut en bas** et de gauche à droite ; un autre sens est indiqué par une **flèche**
commentaire	**Commentaire** explication pour le lecteur ; il n'est pas exécuté		

3 — RÈGLES DE TRACÉ : ERREURS À ÉVITER

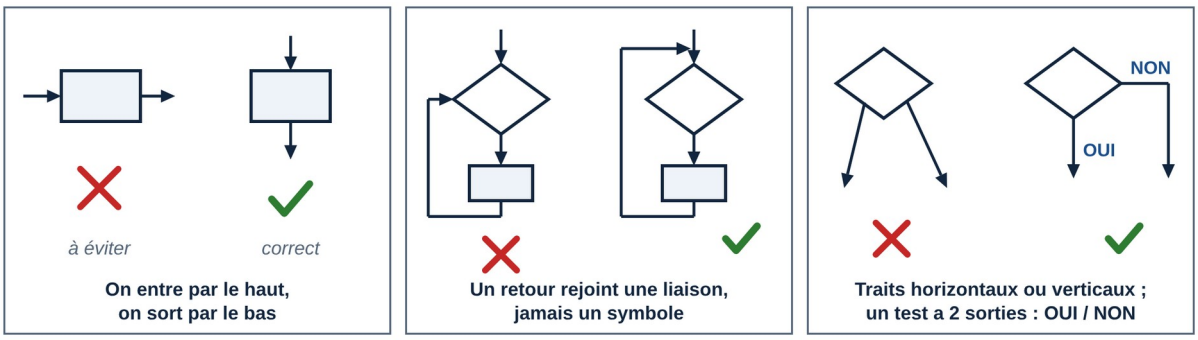


Figure 1 : règles de tracé d'un algorigramme

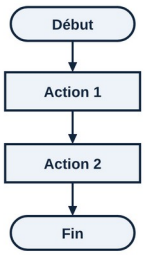
- **Vérifier** que chaque test a exactement **deux sorties**, nommées **OUI** et **NON**.
- **Tracer** uniquement des traits horizontaux ou verticaux ; une boucle revient **sur une liaison**, jamais directement sur un symbole.
- **Commencer** par un seul symbole Début et **terminer** par un symbole Fin.

Cours S2 Algorithmes et algorigrammes · © Henri Turpin et Julie Turpin · Lycée Jean-Claude Fruteau · Sciences de l'Ingénieur

4 — LES STRUCTURES DE BASE

Structure linéaire

Les actions sont exécutées **successivement**, dans l'ordre de leur énoncé.



Algorithme

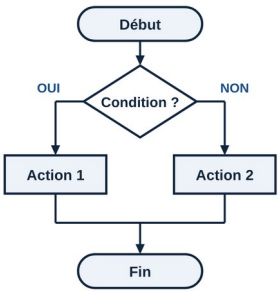
```
Début
  Action 1
  Action 2
Fin
```

En Python

```
# début
action1()
action2()
# fin
```

Structure alternative (Si... Alors... Sinon)

Choix entre **deux séquences qui s'excluent** : selon la condition, on fait l'une **ou** l'autre.



Algorithme

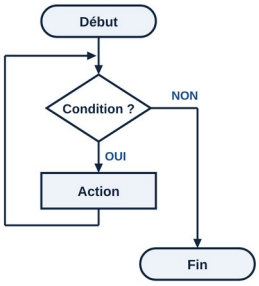
```
Début
  Si condition
    Alors Action 1
    Sinon Action 2
  Fin Si
Fin
```

En Python

```
# début
if condition:
    action1()
else:
    action2()
```

Répétitive « Tant que »

L'action est **répétée tant que** la condition est vraie. La condition est testée **avant** chaque passage : si elle est fausse au départ, l'action n'est jamais faite.



Algorithme

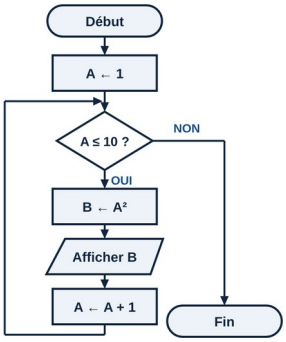
```
Début
  Tant que condition vraie
    Faire Action
  Fin Tant que
Fin
```

En Python

```
# début
while condition:
    action()
```

Répétitive « Pour »

L'action est répétée un **nombre de fois connu** à l'avance ; le compteur augmente de 1 à chaque tour. range(1, 11) donne 1 à 10 (11 exclu).



Algorithme

```
Début
  Pour A de 1 à 10
    B ← A²
    Afficher B
  Fin Pour
Fin
```

En Python

```
# début
for A in range(1, 11):
    B = A ** 2
    print(B)
```


Cours S2 Algorithmes et algorigrammes · © Henri Turpin et Julie Turpin · Lycée Jean-Claude Fruteau · Sciences de l'Ingénieur